2026 EDITION

● ENGINEERING PLAYBOOK

Terminal · Git · GitHub

A complete field guide to the daily loop — branches, pull requests, worktrees and Claude-assisted development.

⇅ Pull Requests ⑂ Worktrees >_ Claude in Terminal

Argeo Alecha
github.com/argeoalecha

BUILDING SCAFFOLDS

↓ BEGIN

CONTENTS

01 Branch & Pull Request
02 PR Management
03 Git Worktrees
04 Claude in Terminal
05 Stash & Recovery
06 Advanced Git Patterns
07 Quick Reference
08 Shell Config
09 Daily Cheatsheet
10 Troubleshooting

Argeo Alecha · 2026 Edition
building scaffolds

01

CORE LOOP

Branch & Pull Request

The core six-step loop. Every feature, fix, or chore lives on a branch — main is always deployable.

1.1 The 6-Step Loop

Every feature, fix, or chore lives on a branch. **Main is always deployable.** PRs are the gate between branch and main.

1 Create a feature branch

```
$ git checkout main && git pull
$ git checkout -b feat/your-feature-name
```

Branch naming convention:

PREFIX	USE FOR	EXAMPLE
`feat/`	New feature or capability	`feat/health-score-tooltip`
`fix/`	Bug fix	`fix/chart-overflow`
`chore/`	Config, deps, refactor	`chore/update-dependencies`
`docs/`	Documentation only	`docs/api-reference`
`hotfix/`	Urgent production fix	`hotfix/auth-crash`

2 Commit on the branch

```
$ git add src/app/dashboard/page.tsx
$ git commit -m "feat(dashboard): add tooltip to health score card"
# Multiple commits are fine — they all roll up into one PR
$ git add .
$ git commit -m "fix(dashboard): correct tooltip z-index"
```

Commit message format: `type(scope): short imperative description`

3 Push the branch

```
$ git push -u origin feat/your-feature-name
# Subsequent pushes — no flags needed
$ git push
```

Vercel auto-preview

The moment this lands on GitHub, Vercel builds a preview URL automatically. No configuration needed — it is wired to the repo.

URL rotabrain-blades-git-feat-name-argeos-projects.vercel.app

4 Open a PR via GitHub CLI

```
$ gh pr create \
    --title "feat: add tooltip to health score card" \
    --body "## What\nAdds a tooltip showing raw score.\n\n## Why\nUsers confused about sca
# Or open in browser
$ gh pr create --web
```

5 Verify on the preview URL

Click through all changed areas. The preview is a full live environment with real Supabase data — exactly what production looks like. Push another commit if needed; the preview rebuilds automatically.

6 Merge and sync

```
$ gh pr merge --squash    # squash keeps history clean
$ git checkout main && git pull
# Vercel promotes the already-verified build — no rebuild
```

Committed directly to main?

Do not panic. Undo and move it to a branch — never force-push main.

```
$ git checkout -b fix/oops-committed-to-main
$ git push -u origin fix/oops-committed-to-main
# Then open a PR. Never force-push main.
```

02

WORKFLOW

PR Management

GitHub CLI workflows for review and merge — keeping branches current and shipping clean history.

2.1 Reviewing & merging

```
# List all open PRs
$ gh pr list
# Check out a PR locally for review
$ gh pr checkout 42
# View PR status and CI checks
$ gh pr status
$ gh pr view 42
# Approve a PR
$ gh pr review 42 --approve
# Request changes
$ gh pr review 42 --request-changes --body "Fix the N+1 query first"
# Merge strategies
$ gh pr merge 42 --merge    # creates merge commit
$ gh pr merge 42 --squash   # squash into one commit (recommended)
$ gh pr merge 42 --rebase   # rebase on main
```

2.2 Keeping your branch up to date

If main has moved while you were on your branch, **rebase** — it keeps history linear:

```
$ git checkout main && git pull
$ git checkout feat/your-feature-name
$ git rebase main
# Resolve any conflicts, then:
$ git rebase --continue
# Force push because history was rewritten
$ git push --force-with-lease
```

--force-with-lease vs --force

Default to **--force-with-lease** always.

--force-with-lease Safe — fails if someone else pushed to the branch.
--force Destructive — overwrites regardless. Never on shared branches.

2.3 Draft PRs

Open a draft when the branch is not ready for review but you want the Vercel preview running:

```
$ gh pr create --draft --title "WIP: redesign sensor grid"
# Mark ready when done
$ gh pr ready
```

03

PARALLEL WORK

Git Worktrees

Multiple branches, zero stashing. Check out several branches at once in separate directories.

3.1 What worktrees solve

Worktrees let you check out multiple branches simultaneously in separate directories. No stashing, no context switching. Essential when pairing with Claude on one branch while running a different branch for reference.

3.2 Core commands

```
# Add a worktree (creates new branch from main)
$ git worktree add ../rotabrain-feat-tooltip feat/health-score-tooltip
# Add a worktree from an existing remote branch
$ git worktree add ../rotabrain-hotfix origin/hotfix/auth-crash
# List all worktrees
$ git worktree list
# Remove a worktree when done
$ git worktree remove ../rotabrain-feat-tooltip
# Prune stale worktree metadata
$ git worktree prune
```

3.3 Worktree + Claude dual-terminal pattern

The key pattern for pairing Claude in Terminal with parallel development:

```
# Terminal A — main branch running for reference
$ cd ~/projects/rotabrain-blades
$ PORT=3000 npm run dev
# Terminal B — Claude is building on the feature branch
$ cd ~/projects/rotabrain-feat-sensor-grid
$ PORT=3001 npm run dev
# Both live simultaneously — compare in browser side by side
# When Claude is done, push from Terminal B
$ git push -u origin feat/sensor-grid
$ gh pr create --title "feat: sensor grid redesign"
```

Port isolation

Run each worktree dev server on a different port to avoid conflicts. Both are live simultaneously — compare behavior directly in the browser.

Main PORT=3000 npm run dev
Feature worktree PORT=3001 npm run dev

3.4 Directory convention

Keep worktrees as siblings of the main repo for easy navigation:

```
~/projects/
  rotabrain-blades/          # main repo  (main branch)
  rotabrain-feat-tooltip/    # worktree   feat/health-score-tooltip
  rotabrain-feat-sensors/    # worktree   feat/sensor-grid
  rotabrain-hotfix-auth/     # worktree   hotfix/auth-crash
```

04

AI PAIRING

Claude in Terminal

Efficient AI-assisted development patterns — scope tightly, pipe errors, review everything.

4.1 Starting a session

```
# Install Claude Code (one-time)
$ npm install -g @anthropic-ai/claude-code
# Start in the worktree you want help with
$ cd ~/projects/rotabrain-feat-tooltip
$ claude
# Start with an immediate task
$ claude "add a tooltip to the HealthScoreCard showing raw value"
# Pipe build errors directly
$ npm run build 2>&1 | claude "fix all TypeScript errors in this output"
```

4.2 Prompting patterns

Be file-specific

```
# BAD — vague
$ claude "fix the chart"
# GOOD — precise, Claude knows exactly what to touch
$ claude "in src/components/SensorChart.tsx the recharts ResponsiveContainer
overflows on screens < 375px — add minWidth 300 and wrap in overflow-x-auto div"
```

Give Claude the error, not the symptom

```
$ npm run build 2>&1 | claude "fix all TypeScript errors"
$ npx eslint src/ --format compact 2>&1 | claude "fix these lint errors"
$ npx tsc --noEmit 2>&1 | claude "resolve all type errors"
```

Checkpoint before risky operations

```
# Before asking Claude for a large refactor
$ git add . && git commit -m "chore: checkpoint before claude refactor"
# If the result is wrong, roll back instantly
$ git reset --hard HEAD
```

Use /add for multi-file context

```
> /add src/app/dashboard/page.tsx
> /add src/components/HealthScoreCard.tsx
> Refactor the health score logic into a shared hook
```

4.3 Review before committing

Always diff Claude's changes before staging. **This is non-negotiable.**

```
# Review all changes Claude made
$ git diff
# Interactive staging — selectively stage hunks
$ git add -p
# Discard a specific file if wrong
$ git checkout -- src/components/SensorChart.tsx
# Discard everything and start over
$ git reset --hard HEAD
```

Never blind-commit Claude output

Always run **git diff** first. Claude writes good code, but:

- It may touch files you did not expect.
- It may make stylistically wrong choices for your codebase.
- Interactive staging (git add -p) is your surgical instrument.

05

RECOVERY

Stash & Recovery

Saving work in progress and undoing mistakes — with a reflog safety net for the worst case.

5.1 Git stash

```
# Stash everything
$ git stash
# Stash with a label (recommended)
$ git stash push -m "WIP: tooltip styling"
# Stash including untracked new files
$ git stash -u
# List all stashes
$ git stash list
# Apply latest stash and remove from list
$ git stash pop
# Apply specific stash without removing
$ git stash apply stash@{2}
# Drop a stash
$ git stash drop stash@{0}
```

Worktrees > stash

If you find yourself stashing frequently to switch branches, use a worktree instead. Each branch gets its own directory — no stashing, no context switching, no lost work.

5.2 Undo reference

SITUATION	COMMAND	SAFE?
Undo last commit (keep staged)	`git reset --soft HEAD~1`	✓ Yes
Undo last commit (keep unstaged)	`git reset HEAD~1`	✓ Yes
Undo last commit (discard)	`git reset --hard HEAD~1`	⚠ Destructive
Discard all unstaged changes	`git checkout -- .`	⚠ Destructive
Discard one file's changes	`git checkout -- path/to/file`	⚠ Destructive
Revert a pushed commit safely	`git revert <sha>`	✓ Yes — adds a commit
Recover lost commit or stash	`git reflog`	✓ Yes — read-only

git reflog — your safety net

Every HEAD movement is logged. If you `reset --hard` by accident:

```
$ git reflog
# abc1234 HEAD@{3}: commit: feat(dashboard): add tooltip
$ git reset --hard abc1234    # recover to that exact state
```

06

ADVANCED

Advanced Git Patterns

Rebase, cherry-pick, bisect and tags — the power tools for a clean, debuggable history.

6.1 Interactive rebase

Clean up messy commit history before merging:

```
$ git rebase -i HEAD~4
# In the editor:
# pick   abc1234  feat: add tooltip
# squash def5678  fix typo in tooltip
# squash ghi9012  fix tooltip z-index
# reword jkl3456  cleanup
# squash = combine into previous commit
# reword = edit the message
# drop   = delete the commit entirely
```

When to use interactive rebase

- Before opening a PR — squash WIP commits into meaningful units.
- NEVER rebase commits that have been pushed and others are working on.
- Always safe on personal feature branches before merge.

6.2 Cherry-pick

Apply a single commit from one branch to another:

```
$ git cherry-pick abc1234

# Cherry-pick a range

$ git cherry-pick abc1234^..def5678

# Cherry-pick without auto-committing

$ git cherry-pick abc1234 --no-commit
```

6.3 Bisect

Binary search through history to find which commit introduced a bug:

`>_ zsh`

```
$ git bisect start

$ git bisect bad          # current commit is broken

$ git bisect good abc1234     # this commit was working

# Git checks out the middle commit. Test it, then:

$ git bisect good   # or: git bisect bad

# Repeat until Git identifies the culprit

$ git bisect reset  # return to HEAD when done
```

6.4 Tags & releases

`>_ zsh`

```
$ git tag -a v1.2.0 -m "Release: sensor grid redesign"

$ git push origin v1.2.0

# Create a GitHub Release from the tag

$ gh release create v1.2.0 --title "v1.2.0 — Sensor Grid" --notes "What changed"
```

07

REFERENCE

Quick Reference

Tables for daily use — Git commands, GitHub CLI essentials, and the commit type vocabulary.

7.1 Daily Git commands

COMMAND	WHAT IT DOES
`git status`	Show staged, unstaged, and untracked files
`git log --oneline --graph`	Compact, visual commit history
`git diff`	Unstaged changes
`git diff --staged`	Staged changes (about to be committed)
`git branch -a`	All local and remote branches
`git branch -d feat/old`	Delete local branch (safe — won't delete unmerged)
`git push origin --delete feat/old`	Delete remote branch
`git fetch --prune`	Sync remote refs, prune deleted remote branches
`git clean -fd`	Remove untracked files and directories

7.2 GitHub CLI essentials

COMMAND	WHAT IT DOES
`gh pr list`	List open PRs in current repo
`gh pr view`	View current branch PR
`gh pr status`	See PR status and CI checks
`gh pr checks`	List CI check results for current PR
`gh issue list`	List open issues
`gh issue create`	Create a new issue
`gh repo view --web`	Open repo in browser
`gh run list`	List GitHub Actions runs
`gh run view`	View a specific Actions run
`gh run watch`	Stream live output of an Actions run
`gh auth login`	Authenticate with GitHub
`gh auth status`	Check current auth status

7.3 Commit type reference

TYPE	USE FOR
`feat`	New feature
`fix`	Bug fix
`chore`	Build, config, deps, refactor — no prod change
`docs`	Documentation only
`style`	Formatting, whitespace — no logic change
`test`	Adding or fixing tests
`perf`	Performance improvement
`ci`	CI/CD configuration changes
`revert`	Reverting a previous commit

08

ENVIRONMENT

Shell Config

Aliases and global Git config — the keystroke economy that makes the loop fast.

8.1 Zsh aliases

`>_ ~/.zshrc`

```
# Git shortcuts

alias gs="git status"

alias ga="git add"

alias gc="git commit -m"

alias gp="git push"

alias gpl="git pull"

alias gd="git diff"

alias gds="git diff --staged"

alias gl="git log --oneline --graph --decorate -20"

alias gb="git branch -a"

alias gco="git checkout"

alias gcob="git checkout -b"

alias grb="git rebase"

alias gst="git stash"

alias gstp="git stash pop"

alias gf="git fetch --prune"

# Worktree shortcuts

alias gwl="git worktree list"

alias gwa="git worktree add"

alias gwr="git worktree remove"

# Project navigation

alias rotabrain="cd ~/projects/rotabrain-blades"

alias hayah="cd ~/projects/hayah-ai"
```

8.2 Git global config

`>_ ~/.gitconfig`

```
[user]

  name  = Argeo Alecha

  email = your@email.com

[core]

  editor = code --wait

[pull]

  rebase = true                     # git pull --rebase by default

[push]

  default         = current

  autoSetupRemote = true     # auto -u origin on first push

[rebase]

  autosquash = true

[alias]

  lg   = log --oneline --graph --decorate

  st   = status

  cm   = commit -m

  undo = reset HEAD~1

  wipe = reset --hard HEAD

  save = !git add -A && git commit -m "WIP: checkpoint"
```

Apply via CLI

`>_ zsh`

```
$ git config --global pull.rebase true

$ git config --global push.autoSetupRemote true

$ git config --global core.editor "code --wait"
```

09

CHEATSHEET

Daily Cheatsheet

The cut-out page for your terminal session — start of day, the working loop, and PR lifecycle.

9.1 Start of day

`>_ zsh`

```
# Sync main

$ git checkout main && git pull

# Create feature branch

$ git checkout -b feat/your-feature-name

# Or use worktrees for parallel tasks

$ git worktree add ../rotabrain-feat-X feat/task-X

$ cd ../rotabrain-feat-X && claude
```

9.2 Working session loop

`>_ zsh`

```
# 1. Give Claude a specific, scoped task

> in src/components/SensorChart.tsx add a loading skeleton ...

# 2. Review before staging

$ git diff

$ git add -p

# 3. Commit the good parts

$ git commit -m "feat(sensor): add loading skeleton"

# 4. Revert specific file if wrong

$ git checkout -- src/components/SensorChart.tsx

# 5. Push when a logical unit is done

$ git push
```

9.3 PR lifecycle

`>_ zsh`

```
# Open PR

$ gh pr create --title "feat: ..." --body "..."

# Check CI

$ gh pr checks

# Review Vercel preview

$ gh pr view --web

# Merge

$ gh pr merge --squash

# Clean up

$ git checkout main && git pull

$ git branch -d feat/your-feature-name

$ git worktree remove ../rotabrain-feat-your-feature-name
```

10

RECOVERY

Troubleshooting

Common situations with exact fixes — when the loop breaks, this is the lookup table.

10.1 Fixes by problem

PROBLEM	FIX
Merge conflict during rebase	Edit conflict markers → git add . → git rebase --continue
Accidentally pushed to main	git revert <sha> → push the revert → open a PR to explain
Worktree: "already checked out"	git worktree list to find it → git worktree remove → retry
PR CI failing (Vercel build)	gh pr checks to see which step. Run npm run build locally first.
Local branch out of sync	git fetch --prune then git rebase origin/main
Cannot delete branch: unmerged	git branch -D (capital D) to force delete
Stash pop has conflicts	Resolve → git stash drop stash@{0} to clean up
Claude edited wrong file	git checkout -- path/to/file to revert that file only
Lost commits after hard reset	git reflog to find SHA → git reset --hard <sha>
gh CLI: auth expired	gh auth login → follow browser OAuth flow

hayah-*ai*

building scaffolds

Argeo Alecha · github.com/argeoalecha · 2026 Edition

Theme